LOW LEVEL DESIGN (LLD)

ON

Agriculture Quiz/Tutor Bot

Course Name : Generative Ai

Institution Name : Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s) :

S.NO	Student Name	Enrolment No
1.	Pawani Shah	EN22CS301700
2.	Praniti Kshirsagar	EN22CS301726
3.	Pranjal Patidar	EN22CS301730
4.	Prafulla Kumar Verma	EN22CS301715
5.	Ritik Thakur	EN22CS301811

Table of Contents

1. Introduction

1.1 Scope of the document

1.2 Intended Audience

1.3 System overview

2. System Design

2.1 Application Design

2.2 Process Flow

2.3 Information Flow

2.4 Component Design

2.5 Key Design Considerations

2.6 API Catalogue

3. References

1. Introduction

The Agriculture Quiz/Tutor Bot is a **Generative AI-based educational system** designed to enhance learning in the agriculture domain through interactive and intelligent assessments. Leveraging the capabilities of **Large Language Models (LLMs)**, the system goes beyond traditional quiz applications by understanding user responses contextually rather than relying on fixed answers. This allows the bot to evaluate answers more
accurately and provide meaningful, human-like feedback.

The core objective of this project is to create a personalized learning experience where users not only receive scores but also detailed explanations for incorrect responses. By focusing on topics such as
farming tips and agricultural practices, the system helps bridge knowledge gaps and reinforces key concepts. Through the integration of Generative AI, the bot acts as an intelligent tutor, making learning more engaging, adaptive, and effective.

1.1 Scope of the Document

This document provides a detailed low-level design of the Agriculture Quiz/Tutor Bot system, focusing on:

- AI-driven quiz generation and evaluation
- Integration with **Large Language Models (LLMs)** for contextual answer assessment
- Application architecture and execution flow
- User interaction through quiz and chatbot interface
- Backend processing and database management

The scope includes actual implementation details, such as:

- Quiz modules and question management system
- AI-based answer evaluation and explanation generation
- APIs for question retrieval, answer submission, and evaluation

• Data storage for users, questions, and results
• Integration with external **Generative AI services for intelligent feedback

1.2 Intended Audience

- Developers
- Project Evaluators
- Designers
- AI/ML Engineers
- Students working on GenAI applications

1.3 System Overview

The Agriculture Quiz/Tutor Bot is an AI-powered application designed to:

- Conduct quizzes based on farming topics
- Evaluate user responses using a Large Language Model (LLM)
- Provide explanations for incorrect answers
- Create a personalized learning experience The

system focuses on domains like:

- Farming Tips
- Soil Management
- Crop Production
- Irrigation Techniques

2. System Design

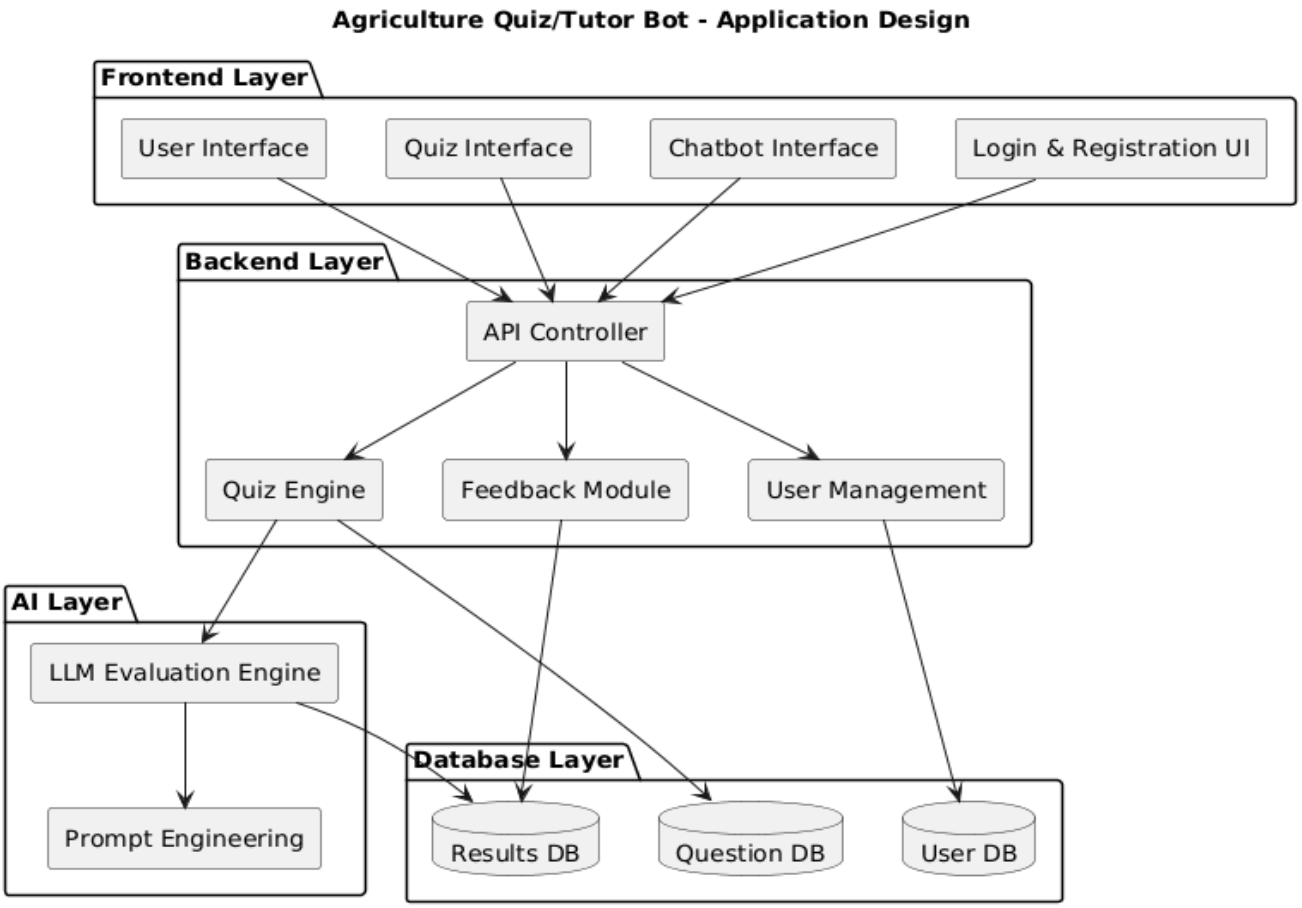
2.1 Application Design

Architecture Type:

Client-Server + AI Integration

Layers:

1. **Frontend Layer**
   - User Interface (Web/App)
   - Quiz display
   - Chatbot interface
2. **Backend Layer**
   - Business Logic
   - Quiz Engine
   - User Management
3. **AI Layer**
   - LLM (for evaluation C explanation)
   - Prompt Engineering module
4. **Database Layer**
   - Stores questions, users, response

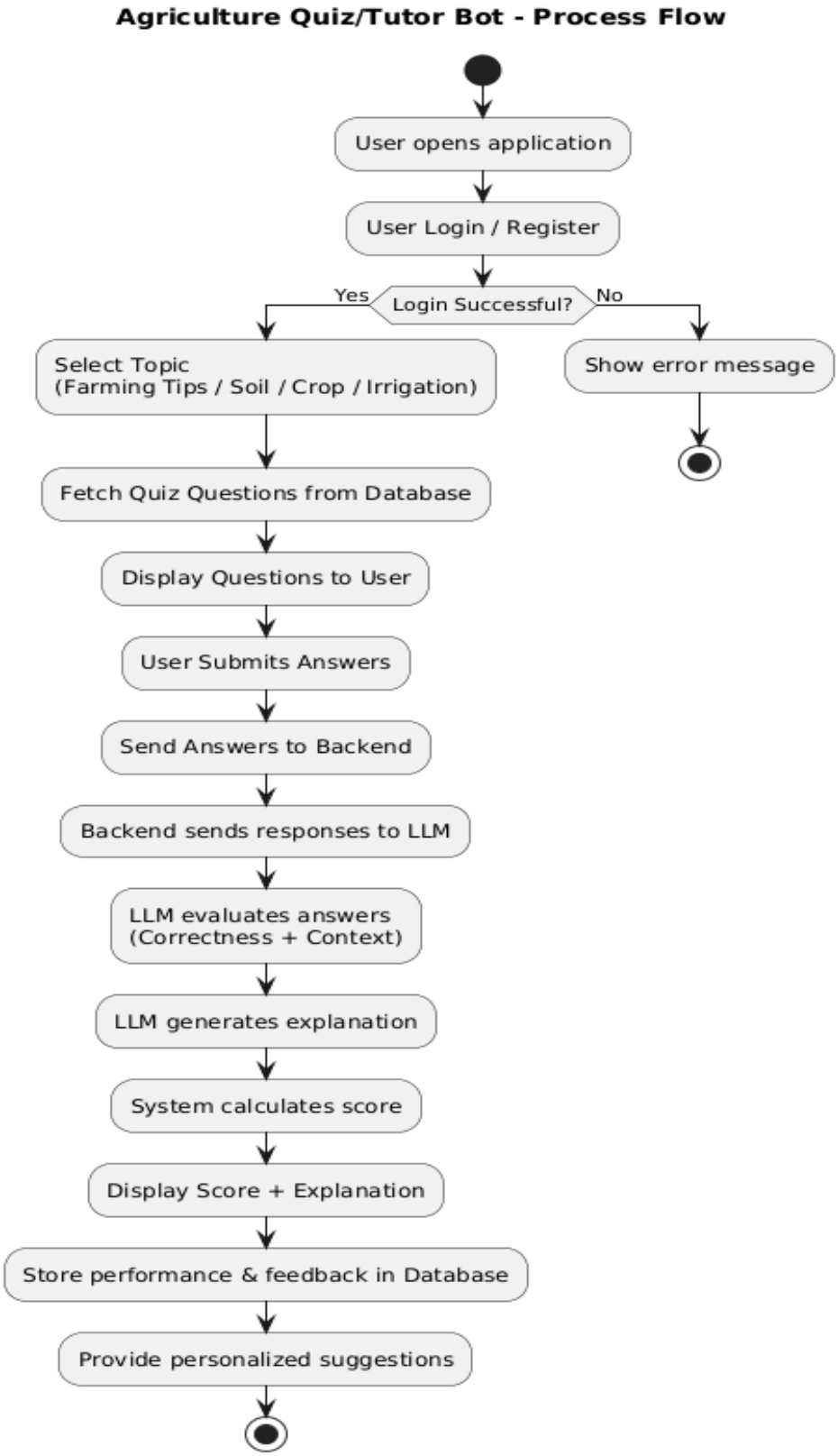


2.2 Process Flow :

Step-by-Step Flow:

1. User logs into the system
2. User selects topic (e.g., Farming Tips)
3. System fetches quiz questions
4. User submits answers
5. Backend sends response to LLM
6. LLM evaluates:
    - Correctness
    - Context
7. LLM generates explanation
8. System displays:
    - Score
    - Explanation

9. Personalized feedback is stored



2.3 Information Flow

Input

- User answers to quiz questions
- Selected topic (e.g., Farming Tips)
- User interaction (text input via chatbot/quiz interface)

Processing

- Backend API processes user responses
- Interaction with Large Language Models (LLMs) for answer evaluation
- Semantic analysis of user input
- Generation of contextual explanations using **Generative AI

Output

- Quiz score (correct/incorrect evaluation)
- Detailed explanation for each answer
- Personalized feedback and suggestions
- Identification of weak topics
- Improved learning recommendations

2.4 Components Design

1. Frontend Components

1. User Interface (UI)

- Displays quiz questions and options
- Shows results, scores, and explanations
- Provides a clean and interactive layout

2. Quiz Interface

- Allows users to select topics (e.g., Farming Tips)
- Handles answer input (text-based responses)
- Navigates between questions

3. Chatbot Interface

- Enables conversational interaction
- Displays AI-generated explanations
- Provides follow-up learning suggestions

4. User Authentication UI

- Login and registration forms
- Profile view and basic user details

2. Backend Components

1. API Layer

- Handles all client requests and responses
- Routes requests to appropriate services
- Endpoints for quiz, user, and evaluation

2. Quiz Management Service

- Retrieves questions from database
- Manages quiz flow and sequencing
- Validates user inputs

3. AI Evaluation Module

- Sends user answers to **Large Language Models (LLMs)**
- Performs semantic evaluation of responses
- Generates contextual explanations using **Generative AI

4. Feedback s Recommendation Module

- Analyzes user performance
- Identifies weak areas
- Provides personalized suggestions

5. User Management Service

- Handles user registration and login
- Manages user profiles and sessions

6. **Database Layer**

- Stores user data
- Stores questions and answers
- Stores quiz results and feedback

2.5 Key Design Considerations

1. **Accuracy**
    - Uses LLM-based semantic evaluation instead of keyword matching
2. **Scalability**
    - Modular and API-driven architecture
3. **Performance**
    - Efficient API handling and quick response generation
4. **Security**
    - Secure authentication and data handling
5. **User Experience**
    - Simple and interactive interface
    - Real-time feedback and explanations

2.6 API Catalogue

1. Get Questions API

- **Endpoint:** /getQuestions
- **Method:** GET
- **Input:** Topic
- **Output:** List of questions

2. Submit Answer API

- **Endpoint:** /submitAnswer
- **Method:** POST
- **Input:**
  - Question ID
  - User Answer
- **Output:**
  - Score
  - Explanation

3. AI Evaluation API

- **Endpoint:** /evaluate
- **Method:** POST
- **Input:**
  - User Answer
  - Correct Answer
- **Output:**
  - Accuracy Score
  - Detailed Explanation

4. User API

- /login
- /register

3. References

- OpenAI / LLM Documentation
- Agriculture Knowledge Sources
- GenAI Course Materials